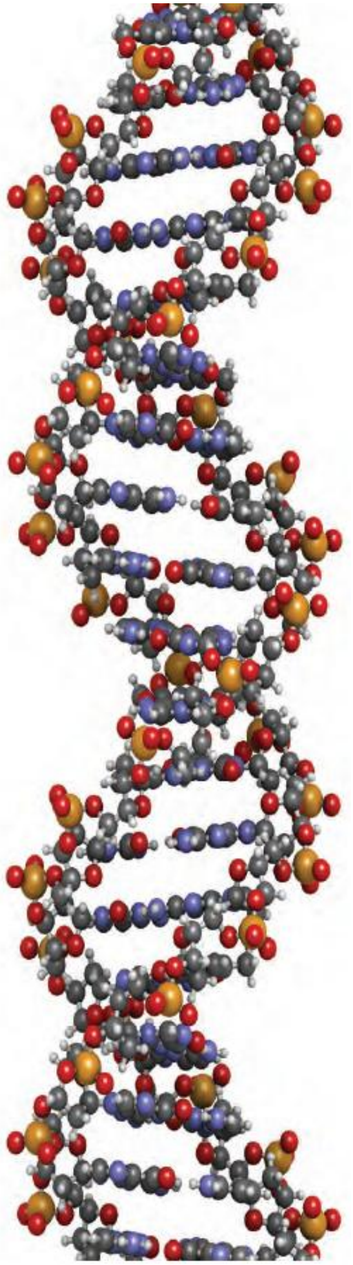
Lecture 3
SEQUENCE ALIGNMENT AND DYNAMIC PROGRAMMING



1. Introduction to Sequence Alignment

Sequence alignment is a fundamental method in bioinformatics used to identify regions of similarity that may indicate functional, structural, or evolutionary relationships between biological sequences (DNA, RNA, or protein).

- **Sequence alignment** is a powerful tool that assesses the similarity of sequences to provide insights into their biological function or evolutionary relationships.
- By identifying regions that **are similar or identical**, sequence alignment demonstrates conserved elements as well as the differences between them.
- Evolution has preserved two broad classes of functional elements in the genome.
  - Such preserved elements are often **homologs**—categorized as either **orthologous** (separated by speciation) or **paralogous** (separated by gene duplication) sequences.
- **Both classes of homologous sequences are genomic sequences descended from a common ancestor.**
- Identifying these elements helps demonstrate the function or evolutionary history of a specific gene sequence.
- Primarily solved using computational methods (most frequently dynamic programming), sequence alignment is an efficient and robust way to identify similarities among genes or genomes.
- **Today's lecture** will discuss the sequence alignment problem, t**he technique of dynamic programming**, and a specific solution to the problem using this technique.

2. Global vs. Local Alignment

Understanding the distinction between these two approaches is crucial for choosing the right tool for your genomic analysis:

- **Global Alignment:** Attempts to align every residue in every sequence from beginning to end. It is most useful when the sequences are of similar length and share a high degree of similarity across their entire length.
- The standard algorithm for this is **Needleman-Wunsch**.

- **Local Alignment:** Focuses on finding the most similar sub-segments within the sequences. This is ideal for sequences that are of different lengths or that share only isolated patches of similarity (e.g., conserved domains or motifs).
- The gold standard for this is the **Smith-Waterman algorithm**.

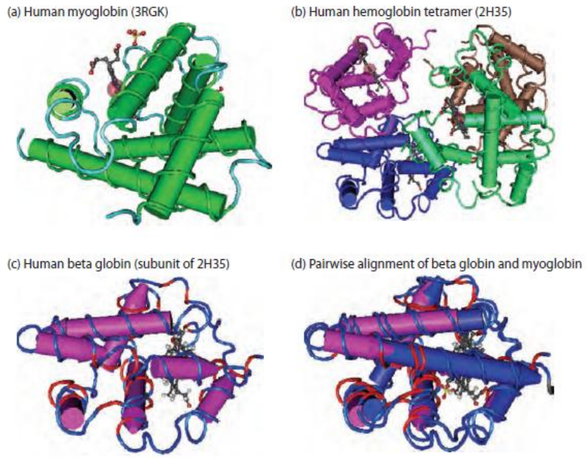
3. The Smith-Waterman Algorithm (Local Alignment)

The Smith-Waterman algorithm uses **dynamic programming** to determine the optimal local alignment. Unlike global alignment, it has a "zero-threshold" which prevents the score from becoming negative, allowing the alignment to restart at any position.

Key features include:

- **Substitution Matrix:** Used to score matches and mismatches (e.g., BLOSUM or PAM for proteins).
- **Gap Penalties:** To account for insertions and deletions (indels).
- **Traceback:** Starts from the highest score in the entire matrix and ends when a cell with a value of zero is reached.

1. Sequence alignment



Definície: Homológia, podobnosť, identita

Príklad: Uvažujme rodinu proteínov globínov. Začíname s ľudským myoglobínom (prístupové číslo NP_005359.1) a beta globínom (prístupové číslo NP_000509.1) ako dvoma proteínmi, ktoré sú vzdialené, ale významne príbuzné. Prístupové čísla sa získavajú z Gene v NCBI. Myoglobín a hemoglobínové reťazce (alfa, beta a iné) sa pravdepodobne rozdelili asi pred 450 miliónmi rokov, približne v čase, keď sa rozdelili línie ľudí a chrupavkovitých rýb.

- **Dve sekvencie sú homológne, ak zdieľajú spoločného evolučného predka.**
- Neexistujú žiadne stupne homológie; sekvencie sú buď homológne, alebo nie.
- Homológne proteíny takmer vždy zdieľajú významne príbuznú trojrozmernú štruktúru. Myoglobín a beta globín majú veľmi podobné štruktúry, ako bolo určené röntgenovou kryštalografiou.
- **Keď sú dve sekvencie homológne, ich sekvencie aminokyselín alebo nukleotidov zvyčajne zdieľajú významnú identitu.**
- **Zatiaľ čo homológia je kvalitatívny záver** (sekvencie sú homológne alebo nie), **identita a podobnosť sú kvantity, ktoré popisujú príbuznosť sekvencií.**
- Je pozoruhodné, že dve molekuly môžu byť homológne bez toho, aby zdieľali štatisticky významnú identitu aminokyselín (alebo nukleotidov). Napríklad v rodine globínov sú všetci členovia homológni, ale niektoré majú sekvencie, ktoré sa tak veľmi rozdelili, že nezdieľajú žiadnu rozpoznateľnú identitu sekvencií (napr. ľudský beta globín a ľudský neuroglobín zdieľajú iba 22 % identitu aminokyselín).
- Vo všeobecnosti sa trojrozmerné štruktúry rozchádzajú oveľa pomalšie ako identita sekvencií aminokyselín medzi dvoma proteínmi. Rozpoznanie tohto typu homológie je obzvlášť náročný bioinformatický problém.

- **Proteíny, ktoré sú homológne, môžu byť ortológne alebo paralógne.**
- **Ortológovia sú homológne sekvencie v rôznych druhoch, ktoré vznikli zo spoločného ancestrálneho génu počas špeciácie.**
- Existuje ľudský gén myoglobínu a gén potkana. Ľudia a hlodavce sa rozdelili asi pred 90 miliónmi rokov (MYA), keď sa jediný ancestrálny gén myoglobínu rozdelil špeciáciou. **Predpokladá sa, že ortológovia majú podobné biologické funkcie; v tomto príklade ľudský a potkaní myoglobín transportujú kyslík v svalových bunkách.**
- **Paralógovia sú homológne sekvencie, ktoré vznikli mechanizmom, ako je duplikácia génu.** Napríklad ľudský alfa 1 globín (NP_000549.1) je paralógny s alfa 2 globínom (NP_000508.1); tieto dva proteíny skutočne zdieľajú 100 % identitu aminokyselín. Ľudský alfa 1 globín a beta globín sú tiež paralógne (ako všetky proteíny zobrazené na obr. hore).
- Všetky globíny majú odlišné vlastnosti, vrátane regionálneho rozloženia v tele, vývojového načasovania génovej expresie a hojnosti. Predpokladá sa, **že všetky majú odlišné, ale príbuzné funkcie ako proteíny prenášajúce kyslík.**

1. Sequence alignment

Jednou z najzákladnejších otázok o géne alebo proteíne je, či je príbuzný s iným génom alebo proteínom.

- **Príbuznosť** dvoch proteínov na úrovni sekvencie naznačuje, že sú homológne.
- **Príbuznosť** tiež naznačuje, že môžu mať spoločné funkcie.
- Analýzou mnohých sekvencií DNA a proteínov je možné identifikovať domény alebo motívy, ktoré sú zdieľané medzi skupinou molekúl. Tieto **analýzy príbuznosti proteínov a génov sa uskutočňujú zarovnávaním sekvencií.**
- S tým, ako dokončujeme sekvenovanie genómov mnohých organizmov, sa úloha zistiť, ako sú proteíny príbuzné v rámci organizmu a medzi organizmami, sa stáva čoraz zásadnejšou pre naše chápanie života.

- **Sequence alignment - párové zarovnanie sekvencií.**
- Pri našom popise toho, **ako môžu byť aminokyseliny (alebo nukleotidy) v dvoch sekvenciách zarovnané a porovnané, budeme vychádzať z evolučnej perspektívy.**

A budeme popisovať algoritmy pre párové zarovnanie.

1. Sequence alignment

	U	C	A	G	
U	UUU Phe UUC UUA Leu UUG	UCU Ser UCC UCA UCG	UAU Tyr UAC Stop UAA - Stop UAG -	UGU Cys UGC Stop UGA - UGG Trp	U C A G
C	CUU Leu CUC CUA CUG	CCU Pro CCC CCA CCG	CAU His CAC CAA Gln CAG	CGU Arg CGC CGA CGG	U C A G
A	AUU Ile AUC AUA Start AUG Met	ACU Thr ACC ACA ACG	AAU Asn AAC AAA Lys AAG	AGU Ser AGC AGA Arg AGG	U C A G
G	GUU Val GUC GUA GUG	GCU Ala GCC GCA GCG	GAU Asp GAC GAA Glu GAG	GGU Glu GGC GGA GGG	U C A G

Zarovnanie proteínov vs. zarovnanie nukleotidov

- Ak máme možnosť rozhodnúť, či budeme zarovnávať sekvenciu DNA alebo sekvenciu proteínu, ktorý kóduje, často je informatívnejšie porovnať sekvenciu proteínu.
- **Existuje na to niekoľko dôvodov.** Mnohé zmeny v sekvencii nukleotidov DNA (najmä na tretej pozícii kodónu) nemenia aminokyselinu, ktorá je špecifikovaná práve **trojpísmenkovým kodónom**. Okrem toho mnohé aminokyseliny zdieľajú príbuzné biofyzikálne vlastnosti (napr. lyzín a arginín sú obe zásadité aminokyseliny).
- Dôležité vzťahy medzi príbuznými (ale nesúladnými) aminokyselinami v zarovnaní je možné zohľadniť pomocou bodovacích systémov.

- Sekvencie nukleotidov DNA sú v tomto ohľade menej informatívne. **Porovnania proteínových sekvencií môžu identifikovať homológne sekvencie**, zatiaľ čo zodpovedajúce porovnania sekvencií DNA (Pearson, 1996) nemajú túto vlastnosť.
- **Napriek tomu je v mnohých prípadoch vhodné porovnávať nukleotidové sekvencie – je to jednoduchšie.**

- Toto porovnanie môže byť dôležité pri potvrdzovaní identity sekvencie DNA pri vyhľadávaní v databáze, pri vyhľadávaní polymorfizmov, pri analýze identity klonovaného fragmentu cDNA, pri porovnávaní regulačných oblastí alebo v mnohých iných aplikáciách.

1. Sequence alignment

Riešenie sekvenčného zarovnávania (Sequence Alignment)

Problém: Chýbajúci predkovia

- Genómy sa v priebehu času neustále menia.
- Kvôli nedostatku vzoriek starovekých genómov je takmer nemožné porovnávať žijúce druhy priamo s ich predkami.
- Sme obmedzení na porovnávanie genómov **súčasných potomkov**.

Cieľ zarovnávania

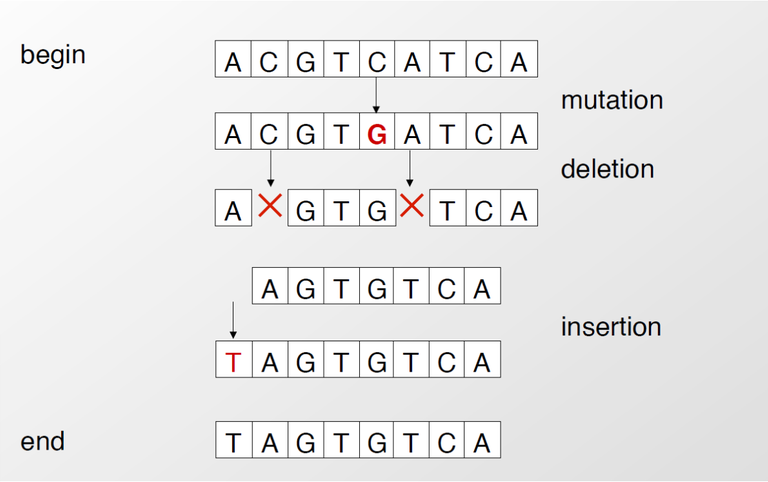
- Cieľom je rekonštruovať (infer) tzv. **„editačné operácie“**, ktoré zmenili genóm v priebehu evolúcie.
- Pracujeme výhradne s koncovými bodmi (súčasnými sekvenciami).

Model a predpoklady

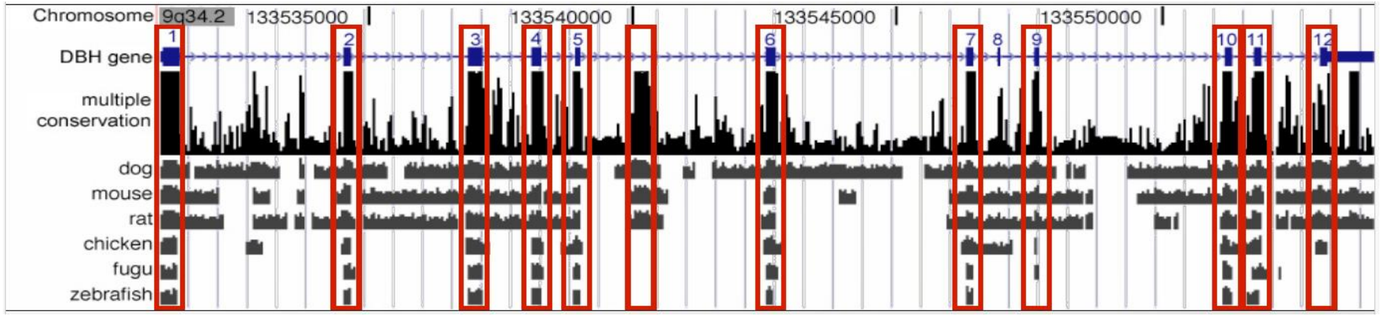
- Biologický problém transformujeme na výpočtovo riešiteľnú úlohu.
- Vyžadujeme model, ktorý je relatívne jednoduchý a spracovateľný (**tractable**).
- Zameriavame sa na tri najčastejšie javy, zatiaľ čo zriedkavejšie udalosti (napr. duplikácie) v tomto modeli ignorujeme.

Tri kľúčové operácie

- **Nukleotidová mutácia:** Zámena jedného nukleotidu za iný v priebehu evolúcie.
- **Nukleotidová delécia:** Odstránenie nukleotidu zo sekvencie.
- **Nukleotidová inzercia:** Pridanie nového nukleotidu do sekvencie.



1. Sequence alignment



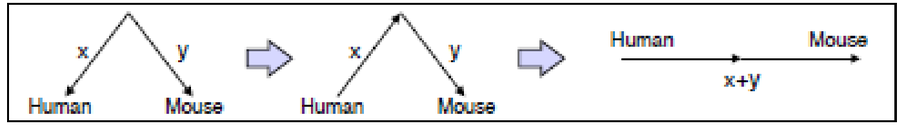
Reverzibilita v čase a modelovanie evolúcie

Predpoklad reversibility

Všetky tri základné operácie (mutácia, delécia, inzercia) sú v našom modeli považované za **vratné**.

- Ak sa nukleotid *N* zmení na *M*, predpokladáme, že je možná aj zmena z *M* na *N*.
- Deléciu nukleotidu možno vrátiť jeho opätovným vložením (re-inzerciou).

Význam pre bioinformatiku



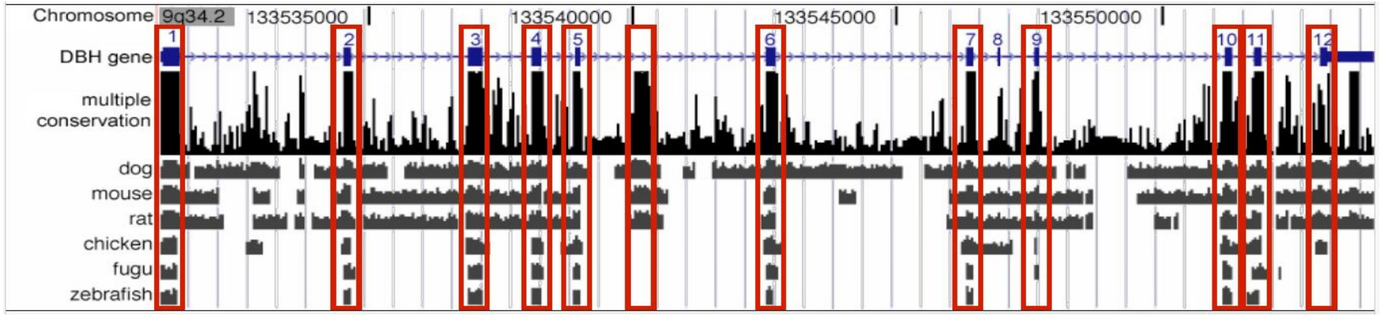
- **Časová reverzibilita** je kľúčový dizajnový predpoklad nášho modelu.
  Umožňuje nám to prepojiť dve súčasne existujúce sekvencie (napr. človek a myš).
- **Postup:** Sledujeme jednu sekvenciu „späť v čase“ k spoločnému predkovi a od neho „vpred“ k druhej sekvencii.
- **Výhoda:** Obchádzame tým problém chýbajúcich sekvencií skutočných predkov.

Model vs. Biologická realita

Dôležité upozornenie

- Časová reverzibilita je **rozhodnutím dizajnéra modelu**, nie vlastnosťou biológie.
- V reálnych biologických systémoch existujú nevratné procesy.
  Príklad: Deaminácia metylovaného cytozínu (*C*) môže viesť k chybnému párovaniu, ktoré nie je reverzibilné.
  Metylovaný CpG sa môže nesprávne párovať s TpG alebo CpA počas replikácie DNA práve z dôvodu deaminácie metylovaného C, ale opačná operácia nemôže nastať

1. Sequence alignment



Kritériá optimality: Ako nájsť „najlepšie“ zarovnanie?

Princíp Occamovej britvy

- Medzi dvoma genómami existuje množstvo možných scenárov zmien.
- Využívame metódu **maximálnej parsimónie** (úspornosti).
- Cieľom je **minimalizovať počet udalostí** (mutácií, inzercií a delécií) potrebných na vysvetlenie rozdielov.

Praktické zohľadnenie pravdepodobnosti

- Nie všetky udalosti sú rovnako pravdepodobné.
- Bodové mutácie sú častejšie než inzercie alebo delécie.
- Tieto nerovnosti v modeli zohľadňujeme pomocou:
  Substitučných matíc (pre mutácie).
  Penalizácií za medzery (gap penalties – pre inzercie a delécie).

Alternatívne prístupy

Okrem parsimónie možno použiť **pravdepodobnostné metódy,** napríklad skryté Markovove modely (**HMM**) alebo Bayesovské metódy.

Dualita: Minimalizácia počtu udalostí (parsimónia) implicitne maximalizuje ich pravdepodobnosť (likelihood).

Dynamic Programming (DP) Theory

Definition:

DP is a powerful method for solving optimization problems by breaking them down into simpler, overlapping sub-structures.

Requirements for DP:

1. **Optimal Substructure**: The optimal solution to the problem contains optimal solutions to its subproblems.
2. **Overlapping Subproblems**: The same subproblems are solved repeatedly throughout the calculation.

DP vs. Greedy Algorithms:

- Greedy algorithms make locally optimal choices that may fail to reach a global optimum.
- DP ensures a **globally optimal solution** by exploring all possible paths systematically.

Why Use DP for Sequence Alignment?

Alignment as a DP Problem:

- **Optimal Substructure**: The best alignment for two sequences of length ***i*** and ***j*** depends on the best alignments of their prefixes (***i-1, j-1***, etc.).
- **Overlapping Subproblems**: Many different possible alignments share the same sub-alignments in their matrices.

Explainability and Transparency:

- Unlike "black-box" models, DP provides a clear, traceable path (the **traceback**) from the final score to the specific biological edits.
- Every score in the matrix is justified by the explicit substitution matrix and gap penalties chosen.

The Duality:

- Minimal event paths in a parsimony framework (DP) are mathematically equivalent to maximizing the likelihood of those events in a probabilistic framework.

Dynamické programovanie (DP)

Úvod do metódy

- Dynamické programovanie je všeobecná a výkonná metóda na riešenie problémov so špecifickou štruktúrou.
- Často sa využíva pri **optimalizačných úlohách**, kde hľadáme najlepšie možné riešenie z viacerých alternatív.

Teoretické predpoklady (Kedy použiť DP?)

Metódu možno úspešne aplikovať, ak problém vykazuje tieto dve vlastnosti:

1. **Optimálna podštruktúra:** Optimálne riešenie celého problému v sebe zahŕňa optimálne riešenia jeho čiastkových podproblémov.
2. **Prekrývajúce sa podproblémy:** Existuje obmedzený počet podproblémov, ktoré sa v rámci výpočtu opakovane vyskytujú.

DP verzus nenásytné (Greedy) algoritmy

- **Greedy algoritmy:** Robia lokálne optimálne rozhodnutia v nádeji, že povedú k globálnemu maximu. Sú však použiteľné na menšiu triedu problémov.
- **Dynamické programovanie:** Na rozdiel od nenásytných prístupov funguje aj tam, kde **lokálne optimálna voľba nemusí viesť ku globálne optimálnemu výsledku**.
- DP systematicky prechádza priestor riešení a garantuje nájdenie globálneho optima tým, že si ukladá výsledky už vyriešených podúloh.

Príprava problému, ak chceme použiť metódy dynamického programovania

Proces nastavenia DP vyžaduje definovanie piatich kľúčových pilierov, ktoré transformujú biologický problém na výpočtový:

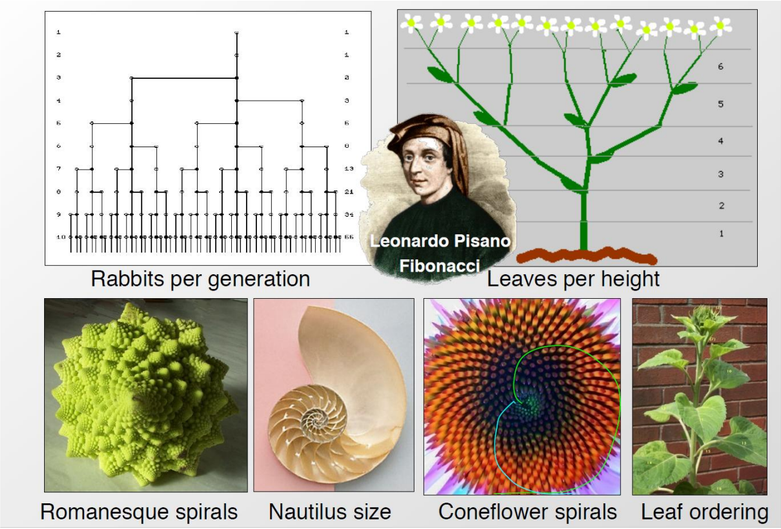
- **Parametrizácia matice:** Určenie rozmerov a premenných matice, ktorá reprezentuje problém.
- **Polynomiálny priestor:** Zabezpečenie, aby počet podproblémov rástol lineárne alebo mocninovo, nie exponenciálne.
  - *Poznámka:* Ak sa podproblémy využívajú opakovane, využíva sa **memoizácia** (ukladanie výsledkov).
- **Poradie prechádzania:** Stanovenie efektívneho poradia výpočtu tak, aby boli výsledky podproblémov pripravené v momente, keď sú potrebné.
- **Rekurzívny vzorec:** Definícia funkcie, ktorá rieši väčší problém na základe jeho menších častí.
- **Ukladanie rozhodnutí:** Uchovanie informácií o voľbách (minimalizácia/maximalizácia) pomocou smerovníkov (pointers) v matici.

Výpočet a hľadanie optimálneho riešenia

Po nastavení rámca nasleduje samotná realizácia, ktorá vedie k výslednému zarovnaniu:

- **Systematické vypĺňanie tabuľky:** Na základe rekurzívneho vzorca sa postupne vypočítajú skóre pre všetky bunky matice.
- **Záznam smerovníkov (Traceback pointers):** Súbežne s výpočtom skóre sa ukladajú smerovníky, ktoré označujú cestu k danému výsledku.
- **Spätné sledovanie (Traceback):** Proces začína pri optimálnom skóre a pomocou smerovníkov spätne rekonštruuje ideálnu cestu (riešenie).
- **Výsledok v bioinformatike:** Pri zarovnávaní sekvencií nám tento proces odhalí presnú sériu editačných operácií, ktoré spájajú dve sekvencie.

2. Dynamic Programming



Fibonacciho čísla poskytujú jednoduchý príklad výhod dynamického programovania.

Fibonacciho postupnosť je rekurzívne definovaná ako

F_0 = F_1 = 1

F_n = F_{n-1} + F_{n-2} pre n >= 2.

Vyvinieme algoritmus na výpočet n-tého Fibonacciho čísla, ktorý následne vylepšíme – najprv pomocou memoizácie a neskôr pomocou dynamického programovania, aby sme ilustrovali kľúčové koncepty.

F6=F5+F4=(F4+F3)+(F3+F2))=….=(3+2)+(2+1)=5+3=8

Výpočet Fibonacciho čísel: Zhora nadol

Cieľ:

Vypočítať n-té Fibonacciho číslo.-

F(0)=1,

F(1)=1,

F(n)=F(n-1)+F(n-2)

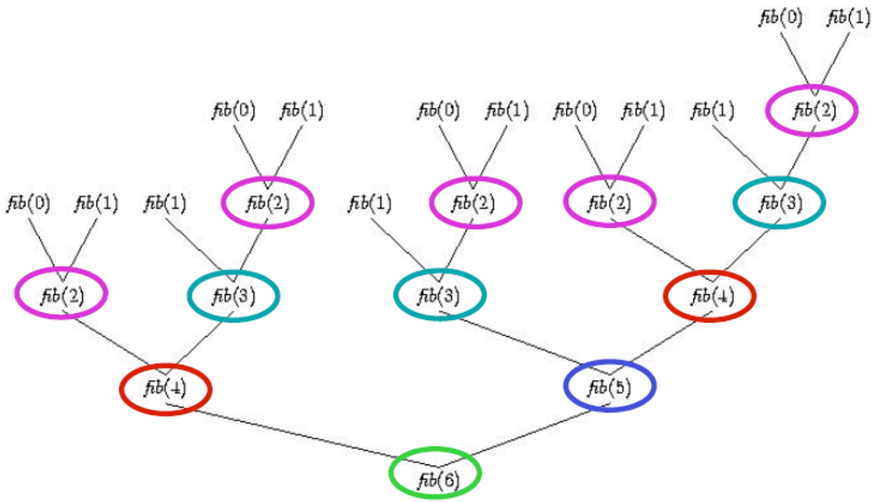
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377,…

Fibonacciho čísla sú definované rekurzívne (Python):

```
# Assume n is a non-negative integer.
def fib(n):
    if n == 0 or n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
```

Analýza:

T(n) = T(n-1) + T(n-2) = (...) = O(2^n)



„Riešenie pomocou memoizácie

Lepším riešením, ktoré stále využíva prístup zhora nadol (top-down), je memoizácia odpovedí na jednotlivé podproblémy.“

- **Prístup zhora nadol (Top-down):** Znamená, že začínate hlavným problémom a rekurzívne ho rozkladáte na menšie časti.
- **Memoizácia:** Je technika, pri ktorej si algoritmus „zapamätá“ (uloží do pamäte) výsledok podproblému hneď, ako ho prvýkrát vypočíta. Ak na ten istý podproblém narazí znova, výsledok už nepočíta, ale jednoducho ho vytiahne z pamäte.

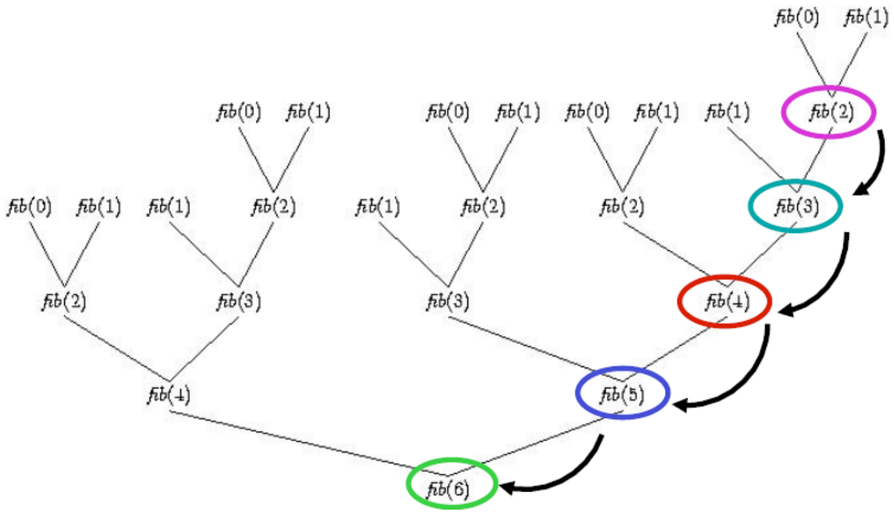
fib_table	
F[1]	1
F[2]	1
F[3]	2
F[4]	3
F[5]	5
F[6]	8
F[7]	13
F[8]	21
F[9]	34
F[10]	55
F[11]	89
F[12]	?

Výpočet Fibonacciho čísel: memoizácia

```
# Assume n is a non-negative integer.
fibs = {0: 1, 1: 1} # stores subproblem answers
def fib(n):
    if n not in fibs:
        x = fib(n - 2)
        y = fib(n - 1)
        fibs[n] = x + y
    return fibs[n]
```

Cieľ:
Vypočítať n-té Fibonacciho číslo.-
F(0)=1,
F(1)=1,
F(n)=F(n-1)+F(n-2)
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377,…

Analýza:
-***T(n) = O(n)***
Note that this implementation now runs in T(n) = O(n) time because each subproblem is computed at most once.



Riešenie pomocou dynamického programovania (DP)

Namiesto začínania od $F(n)$ (rekurzia) začíname od základných prípadov (zdola nahor), pretože vieme, že budeme potrebovať všetky podproblémy.

Aplikácia 5 krokov DP na Fibonacciho čísla:

1. **Parametrizácia matice**: Matica je jednorozmerná; podproblém má len jeden parameter $F(x)$.
2. **Polynomiálny priestor**: Máme iba $n-1$ podproblémov, čo predstavuje polynomiálny priestor.
3. **Poradie prechádzania**: Použijeme prístup zdola nahor (**bottom-up**) – podproblémy riešime vzostupne.
4. **Rekurzívny vzorec**: Použijeme známy vzťah $F(n) = F(n-1) + F(n-2)$.
5. **Ukladanie rozhodnutí**: V tomto prípade netreba ukladať žiadne voľby, keďže vzorec neobsahuje minimalizáciu ani maximalizáciu.

Výsledok: Výpočet n-tého čísla dosiahneme v lineárnom čase $O(n)$.

```python
# Assume n is a non-negative integer
def fib(n):
    x = y = 1
    for i in range(1, n):
        x, y = y, x + y
    return x
```

DP vs. Memoizácia – Kedy a čo použiť?

Hoci oba prístupy eliminujú opakovanú prácu, líšia sa v spôsobe vykonávania a efektivite.

Porovnanie prístupov:

Memoizácia (Top-down):
- Užitočná vtedy, keď nie je potrebné vypočítať všetky podproblémy.
- Zachováva logiku rekurzie, čo môže niesť réžiu spojenú s volaním funkcií.

Dynamické programovanie (Bottom-up):
- Preferované vtedy, keď sa vyžadujú riešenia všetkých podproblémov.
- Šetrí systémovú réžiu rekurzívnych volaní.

Optimalizácia priestoru:
V prípade Fibonacciho čísel stačí konštantný priestor (pamätáme si len posledné dve hodnoty), zatiaľ čo všeobecné DP riešenia zvyčajne vyžadujú celú tabuľku pre opakované použitie výsledkov.

Čo hovorí Dynamické programovanie v teoretickej rovine

- Charakteristické znaky dynamického programovania
- Optimálna podštruktúra: Optimálne riešenie problému (inštancia) obsahuje optimálne riešenia čiastkových problémov
- Prekrývajúce sa podproblémy: Obmedzený počet rôznych podproblémov, ktoré sa opakujú mnohokrát

Typická situácia pre optimalizačné problémy (na rozdiel od príkladu Fib)

- Optimálna voľba vykonaná lokálne: max(skóre podriešení)
- Skóre sa typicky sčítava v priestore hľadania
- Spätné sledovanie bežné, nájdenie optimálnej cesty sa realizuje z jednotlivých volieb

Ideálna je stredná cesta (podľa rozsahu náročnosti)

- **Ľahšie:** Greedy choice možná v každom kroku
- **Dynamické programovanie:** vyžaduje spätné sledovanie, aby sa našla táto optimálna cesta
- **Ťažšie:** nemáme celkovú optimálnu cestu, napr. závislosti od čiastkových problémov

2. Dynamic Programming

Dynamické programovanie v praxi

Nastavenie dynamického programovania

- **Nájdite parametrizáciu „matice“ (počet dimenzií, premenné)**
- **Uistite sa, že priestor podproblémov je konečný! (nie exponenciálny)**
  - **Ak sa nepoužívajú všetky podproblémy, je lepšie použiť memoizáciu**
    - Memoizácia je technika, pri ktorej sa ukladajú výsledky volaní funkcií a vracia sa uložený výsledok, keď sa rovnaké vstupy vyskytnú znova.
  - **Ak opätovné použitie nie je rozsiahle, možno dynamické programovanie nie je správne riešenie!**
    - Dynamické programovanie je najefektívnejšie, keď sa podproblémy opakovane používajú. Ak sa podproblémy používajú len zriedka, iné algoritmy môžu byť efektívnejšie.
- **Poradie prechádzania: čiastkové výsledky sú pripravené, keď ich potrebujete**
  - Vysvetlenie: Určite, v akom poradí budete vypĺňať maticu, aby ste mali výsledky podproblémov k dispozícii, keď ich budete potrebovať na výpočet väčších podproblémov.
  - **Na poradí výpočtu záleží! (zdola nahor, ale nie vždy zrejmé)**
    - Zvyčajne sa matica vypĺňa zdola nahor, ale v niektorých prípadoch môže byť potrebné iné poradie.
- **Rekurzívny vzorec: väčšie problémy = F(čiastkové časti)**
  - Vysvetlenie: Nájdite vzorec, ktorý definuje, ako vypočítať riešenie väčšieho problému z riešení menších podproblémov.
- **Pamätajte si možnosti: zvyčajne F() zahŕňa min() alebo max()**
  - Vysvetlenie: Vzorec často zahŕňa výber najlepšej možnosti (napr. minimálne alebo maximálne skóre).
  - **Potrebujete reprezentáciu na ukladanie ukazovateľov, je to polynomiálne?**
    - Ukazovatele sa používajú na sledovanie najlepších možností, ktoré boli vybraté. Je dôležité, aby množstvo pamäte potrebnej na ukladanie týchto ukazovateľov rástlo polynomiálne, nie exponenciálne.
- **Potom začnite počítať**
  - **Systematicky vyplňte tabuľku výsledkov, nájdite optimálne skóre**
    - Vysvetlenie: Vypočítajte a uložte riešenia všetkých podproblémov do matice.
  - **Sledujte späť z optimálneho skóre, nájdite optimálne riešenie**
    - Vysvetlenie: Po vypočítaní optimálneho skóre sledujte späť kroky, ktoré viedli k tomuto skóre, aby ste našli optimálne riešenie.

Sequence Alignment using Dynamic Programming
Dynamické programovanie pre zarovnanie sekvencií

- **Aditívne skóre, budovanie riešenia z menších častí**
- **Prefixová matica: konečné podproblémy, exponenciálne cesty**

  Vysvetlenie: Matica sa používa na ukladanie skóre zarovnania pre všetky prefixy (začiatky) sekvencií. Hoci existuje exponenciálne veľa možných zarovnaní, existuje len konečný počet prefixov.
- **Dualita:**

  každý záznam → skóre prefixového zarovnania;
  cesta → zarovnanie

  Vysvetlenie: Každý záznam v matici predstavuje skóre zarovnania pre prefixy sekvencií, ktoré zodpovedajú riadku a stĺpcu. Cesta cez maticu predstavuje konkrétne zarovnanie sekvencií.

Today's goal:

How do we actually align two genes?
Ako vlastne zarovnáme dva gény?

- **Formulácia 1: Najdlhší spoločný podreťazec (Longest Common Substring)**
- **Formulácia 2: Najdlhšia spoločná podpostupnosť (LCS) – povolíme medzery**
- **Formulácia 3: Zarovnávanie sekvencií ako editačná vzdialenosť (Edit Distance)**
- **Formulácia 4: Modely s premenlivou cenou medzier (Gaps)**
- **Formulácia 5: Problém enumerácie: Prečo „hrubá sila“ zlyháva?**

Formulácia problému: Od jednoduchosti k realite

Úvod do modelovania

- Cieľom tejto sekcie je iteratívne zvyšovať komplexnosť problému, až kým nedosiahneme model verný sekvenčnému zarovnávaniu.
- Tento proces slúži ako príprava na pochopenie **Needleman-Wunschovho algoritmu**.

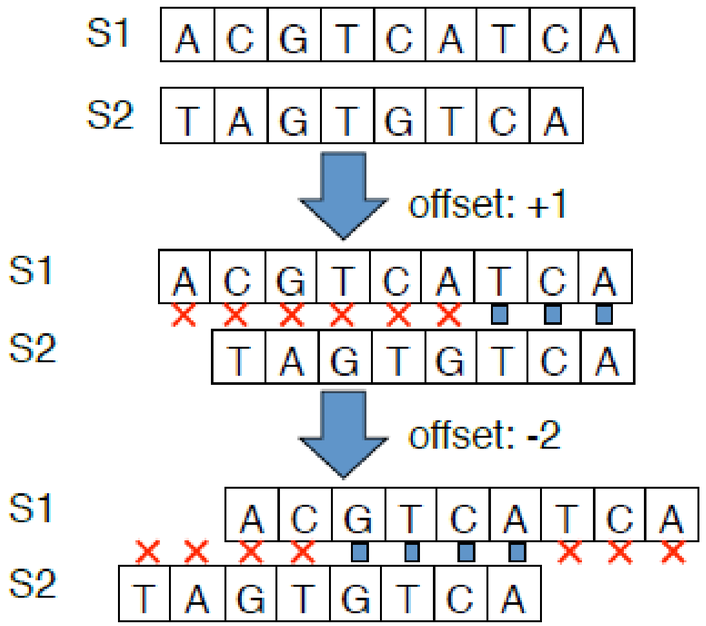
Formulácia 1: Najdlhší spoločný podreťazec (Longest Common Substring)

- **Prístup**: Nukleotidové sekvencie vnímame ako reťazce nad abecedou {A, C, G, T}.
- **Definícia**: Hľadáme najdlhší súvislý úsek, ktorý je identický v oboch reťazcoch (S_1 a S_2).
- **Obmedzenie**: V tejto prvej formulácii nie sú povolené **žiadne medzery (gaps)** v rámci podreťazcov.

Príklad a efektivita

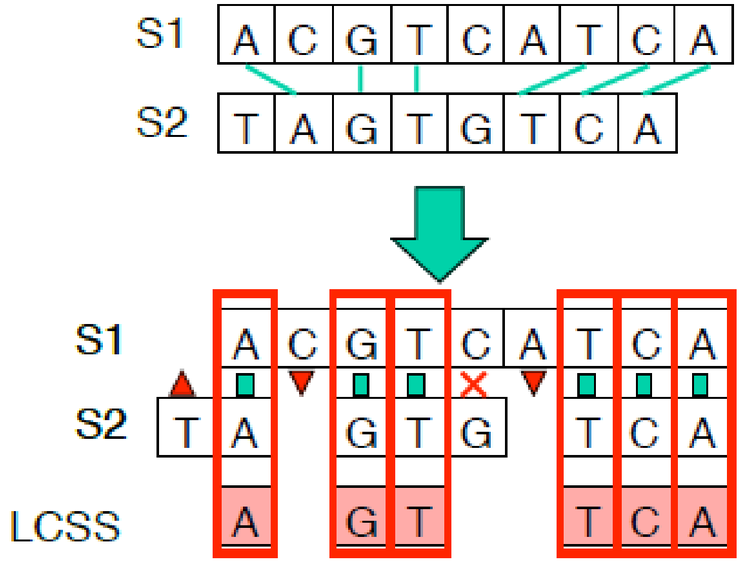
- Ak S_1 = {ACGTCATCA} a S_2 = {TAGTGTCA},
  tak najdlhším spoločným podreťazcom je **GTCA**.
- Algoritmus by skúšal rôzne posuny (offsets) sekvencií a hľadal najdlhšiu zhodu.

Nevýhoda: Výpočtová zložitosť je kvadratická voči dĺžke kratšej sekvencie, čo je na takýto jednoduchý problém príliš pomalé.



Formulácia 2: Najdlhšia spoločná podpostupnosť (LCS) – povolíme medzery

- **Rozšírenie modelu**:
  Na rozdiel od podreťazcov (substrings) táto formulácia povoľuje v sekvenciách **medzery (gaps)**.
- **Definícia podpostupnosti**:
  Z je podpostupnosťou ***X***, ak existuje rastúca postupnosť indexov taká, že prvky ***Z*** sa v presnom poradí nachádzajú v ***X***, ale nemusia nasledovať bezprostredne za sebou.
- **Cieľ problému**:
  Pre dve sekvencie ***X*** a ***Y*** hľadáme spoločnú podpostupnosť ***Z*** s maximálnou možnou dĺžkou.



Príklad porovnania

- **Sekvencie**: S_1 = {ACGTCATCA}, S_2 = {TAGTGTCA}.
- **Najdlhší spoločný podreťazec (bez medzier)**: GTCA (dĺžka 4).
- **Najdlhšia spoločná podpostupnosť (s medzerami)**: **AGTTCA** (dĺžka 6).

Záver: Povolenie medzier umožňuje nájsť oveľa rozsiahlejšie zhody, čo lepšie reflektuje biologickú realitu evolučných zmien.

3. Sequence alignment

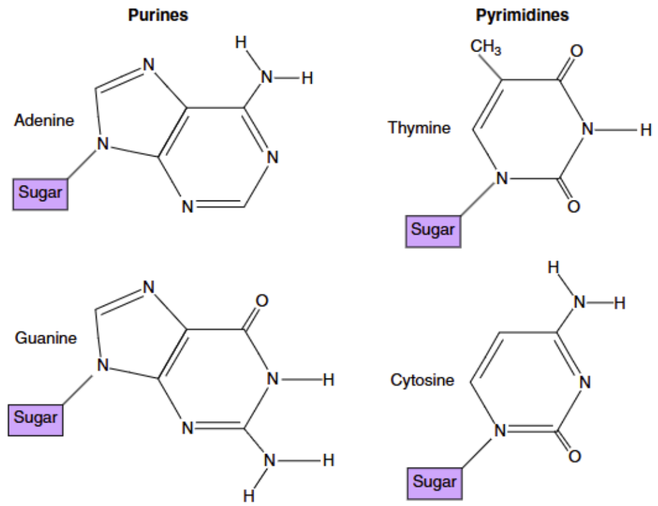
Formulácia 3: Zarovnávanie sekvencií ako editačná vzdialenosť (Edit Distance)

- **Prechod od LCS**: Predchádzajúca formulácia (LCS) nerozlišovala medzi rôznymi typmi editačných operácií a predpokladala, že všetky sú rovnako pravdepodobné.
- **Biologická realita**: Keďže substitúcie sú v prírode oveľa častejšie ako inzercie a delécie, model musíme doplniť o **cenové funkcie** (cost functions), ktoré uprednostňujú substitúcie.
- **Editačná vzdialenosť**: Zarovnávanie sekvencií definujeme ako špeciálny prípad klasického problému „Edit-Distance“ z informatiky.

Biologické zdôvodnenie skórovania

- **Štruktúra báz**: Nukleotidy delíme na **puríny** (A, G – väčšie, dva kruhy) a **pyrimidíny** (C, T – menšie, jeden kruh).
- **Chyby polymerázy**: DNA polymeráza oveľa pravdepodobnejšie zamení dva puríny alebo dva pyrimidíny kvôli ich podobnej štruktúre.
- **Symetria matice**: Skórovacia matica je **symetrická**, čo priamo podporuje náš predpoklad o **časovej reverzibilite** modelu.

Poznámka: Tento prístup pomocou skórovacích matíc je kľúčový pre vašu požiadavku na **vysvetliteľnosť**. Na rozdiel od „black-box“ modelov tu presne definujeme, prečo má určitá mutácia konkrétnu váhu na základe biologických vlastností purínov a pyrimidínov.



	A	G	T	C
A	+1	-½	-1	-1
G	-½	+1	-1	-1
T	-1	-1	+1	-½
C	-1	-1	-½	+1

Formulácia 3: Zarovnávanie sekvencií ako editačná vzdialenosť (Edit Distance)
Interpretácia skóre a zložitosť algoritmu

Dualita skórovania
Pri nastavovaní skórovania v bioinformatike neustále balansujeme medzi dvoma svetmi:

- **Pravdepodobnostný pohľad (biologická pravdepodobnosť):**
  Ako často sa mutácia alebo delécia reálne deje v bunke?
- **Algoritmický pohľad (algoritmické skóre):** Aké číslo priradíme danej **operácii v našej matici, aby sme našli matematické minimum?**

	A	G	T	C
A	+1	-½	-1	-1
G	-½	+1	-1	-1
T	-1	-1	+1	-½
C	-1	-1	-½	+1

Cieľ:
Nájsť **najlacnejšiu postupnosť operácií** (podľa nákladovej matice), ktorá transformuje počiatočnú sekvenciu na finálnu.

Komplexita editačnej vzdialenosti

- **Editačná vzdialenosť:** Metrika merajúca minimálny počet zmien (edícií) potrebných na premenu jedného reťazca na druhý.
- **Teoretický limit O(n^2):** Existuje dôkaz (Backurs a Indyk, 2015), že vo všeobecnom prípade nie je možné tento problém vyriešiť rýchlejšie ako v kvadratickom čase.
- **Súvis s SETH:** Tento limit vychádza zo „Silnej hypotézy o exponenciálnom čase“ (SETH), ktorá predpokladá, že NP-úplné problémy nemožno v najhoršom prípade riešiť v subexponenciálnom čase.
  Poznámka: editačná vzdialenosť (známa aj ako Levenshteinova vzdialenosť) sa nedá v všeobecnom prípade vypočítať rýchlejšie ako v čase O(n^2). To znamená, že pre extrémne dlhé genómy potrebujeme hľadať efektívne riešenia, pretože kvadratický čas je teoretické minimum, ktoré nevieme podliezť bez porušenia základných matematických predpokladov (SETH).

Biologická poznámka:

DNA polymeráza: Enzým zodpovedný za kopírovanie vlákna DNA počas replikácie; jeho chybovosť určuje pravdepodobnosti v modeli.

Formulácia 4: Modely s premenlivou cenou medzier (Gaps)

Biologický context

Vytvorenie novej medzery (**gap opening**) je biologicky menej pravdepodobné (drahšie)
než predĺženie už existujúcej medzery (**gap extension**).

Tento fakt zohľadňujeme pomocou rôznych modelov penalizácie:

Typy penalizácií za medzery

- **Lineárna (Linear):** Fixná cena pre každú medzeru bez ohľadu na jej kontext (rovnako ako vo Formulácii 3).
- **Afinná (Affine):** Vysoká počiatočná cena za otvorenie medzery a následne nízka cena za každý ďalší nukleotid v tej istej medzere.
- **Všeobecná (General):** Umožňuje ľubovoľnú cenovú funkciu (pozor: môže zvýšiť asymptotickú zložitosť algoritmu).
- **Špeciálny model: „Frame-aware“ penalizácia**
  V kódujúcich sekvenciách tri nukleotidy (kodón) tvoria jednu aminokyselinu. Indely, ktoré nie sú násobkom 3, narúšajú celý čítací rámec a dramaticky menia výsledný proteín.
  Logika skórovania:
  Medzery s dĺžkou **násobku 3** sú penalizované **menej**, pretože zachovávajú integritu čítacieho rámca a väčšinu aminokyselín.

Formulácia 5: Problém enumerácie: Prečo „hrubá sila“ zlyháva?

Základné problémy:

- **Longest Common Substring**:
  Pri hľadaní najdlhšieho spoločného podreťazca (bez medzier) existuje len ***O(n)*** možných zarovnaní, čo je ľahko vypočítateľné.
- **Zarovnanie s medzerami**:
  Akonáhle do modelu pridáme medzery (gaps), počet možných kombinácií rastie exponenciálne.

Matematické vyjadrenie

Pre dve sekvencie dĺžky ***n*** a ***m*** (kde ***n > m***) je počet možných zarovnaní bez vnútorných medzier (NBA) definovaný ako:

$$\binom{n+m}{m} = \frac{(n+m)!}{n!m!} \approx \frac{(2n)!}{(n!)^2} \approx \frac{\sqrt{4\pi n}\frac{(2n)^{2n}}{e^{2n}}}{(\sqrt{2\pi n}\frac{(n)^n}{e^n})^2} = \frac{2^{2n}}{\sqrt{\pi n}}$$

Prečo potrebujeme lepší algoritmus?

- **Príklad z reality**:
  Pri dvoch sekvenciách s dĺžkou len **1000 nukleotidov** je počet možných zarovnaní vyšší než **počet atómov vo vesmíre**.
  Už pri dĺžke n = 30 je počet kombinácií (> 10^{17}) príliš veľký na to, aby ich súčasné počítače spracovali v rozumnom čase.

Záver: Hrubá sila (Brute-force) je v bioinformatike nepoužiteľná.
Dynamické programovanie je absolútnou nutnosťou, pretože nám umožňuje nájsť
optimálne riešenie bez prechádzania všetkých možností.

4. The Needleman-Wunsch Algorithm

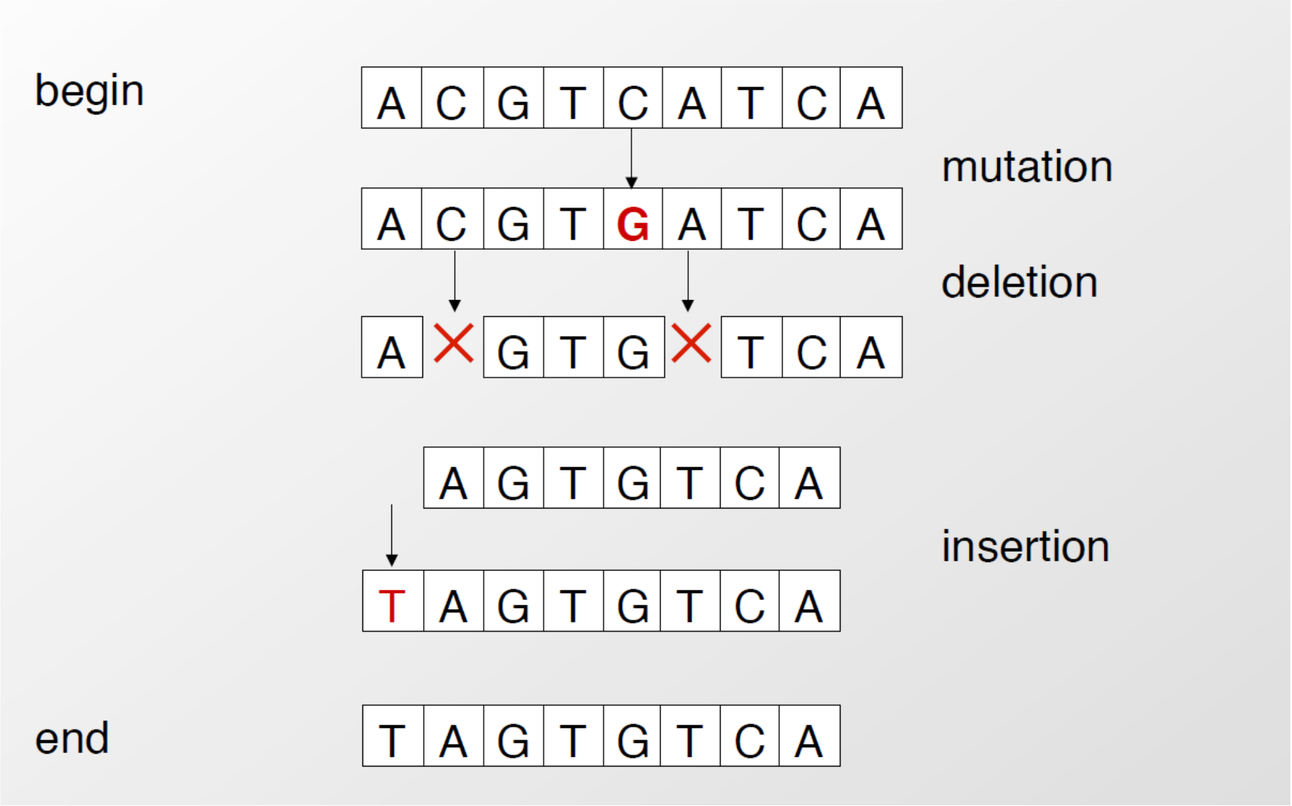
Today's goal:

Needleman-Wunschov algoritmus

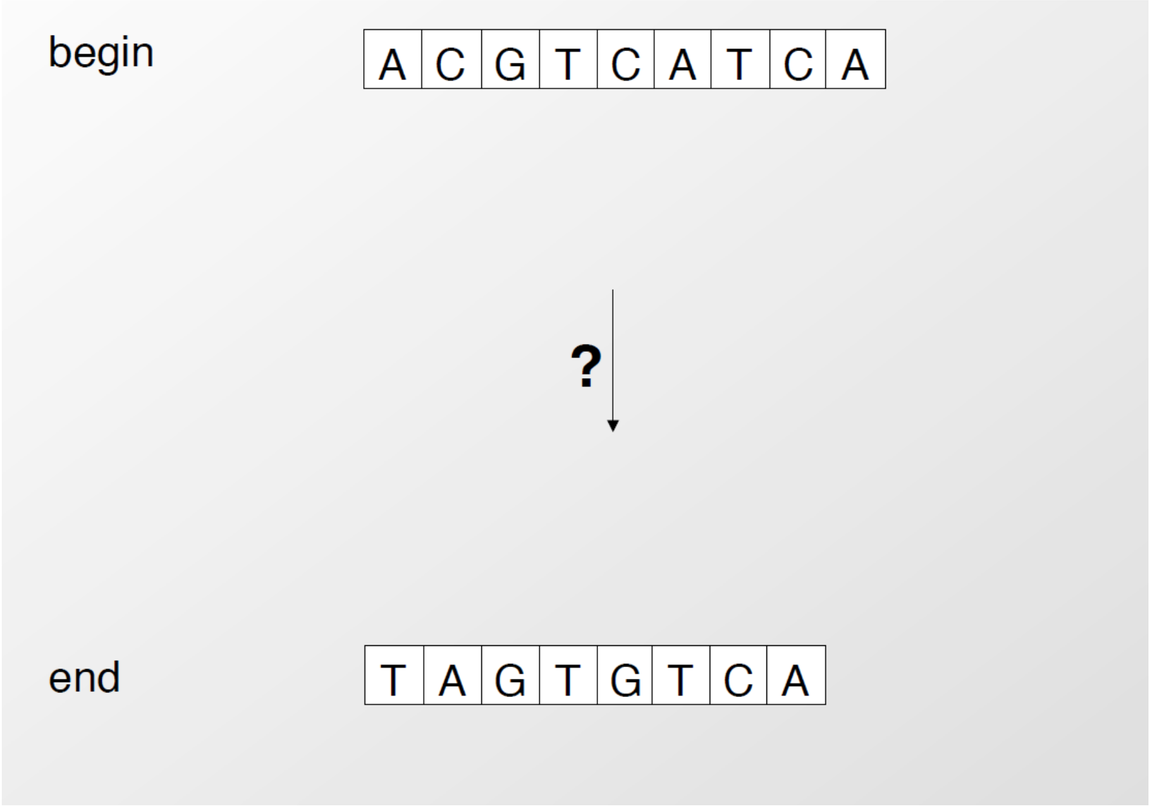
- **ff**

2. Sequence alignment

Genómy sa v priebehu času menia



Cieľ: Odvodiť editačné operácie



4. The Needleman-Wunsch Algorithm

Čo je to?

- Prvý algoritmus využívajúci dynamické programovanie na porovnávanie biologických sekvencií (publikovaný v roku 1970).
- Slúži na hľadanie **globálneho zarovnania**, teda najlepšieho možného priradenia dvoch sekvencií po celej ich dĺžke (od začiatku až do konca).
- Problém rozkladá na podproblémy zarovnania kratších prefixov oboch sekvencií a výsledky ukladá do 2D matice.
- **Biologický význam.**
  Používa sa pri porovnávaní dvoch **úzko príbuzných sekvencií**, ktoré majú podobnú dĺžku a očakávame u nich podobnosť po celej dĺžke (napr. rovnaký gén u dvoch rôznych druhov).

Kľúčové vlastnosti

1. **Garantovaná optimálnosť:** Algoritmus vždy nájde matematicky najlepšie možné zarovnanie vzhľadom na zvolenú skórovaciu schému.
2. **Tri základné komponenty:**
   - **Inicializácia:** Nastavenie okrajových podmienok (penalizácia za úvodné medzery).
   - **Vypĺňanie matice:** Výpočet skóre pre každú bunku na základe troch susedných buniek (zhoda/mutácia, inzercia, delécia).
   - **Traceback:** Spätné sledovanie cesty z pravého dolného rohu matice do ľavého horného, čím sa získa výsledné zarovnanie.
3. **Výpočtová zložitosť:** ***O(n x m)***, kde n a m sú dĺžky sekvencií. To je obrovský skok oproti nepoužiteľnej „hrubej sile“.

4. The Needleman-Wunsch Algorithm

Globálne zarovnávanie a Needleman-Wunschov algoritmus

- Na riešenie zložitého problému všeobecného zarovnávania dvoch reťazcov S (dĺžka ***n***) a T (dĺžka ***m***) využívame **dynamické programovanie**.
- Cieľom nie je len maximalizovať dĺžku zhody, ale nájsť podpostupnosť, ktorá **optimalizuje celkové skóre** definované našou skórovacou funkciou.

Parametre modelu

- **d (Gap penalty):**
  Cena (penalizácia) za vloženie medzery (indel).
- **s(x, y) (Substitution score):**
  Skóre za zarovnanie bázy x s bázou y.
- Tieto hodnoty sa odvodzujú z pravdepodobností mutácií a inzercií/delécií získaných experimentálne alebo analýzou blízko príbuzných sekvencií.

	A	G	T	C
A	+1	-½	-1	-1
G	-½	+1	-1	-1
T	-1	-1	+1	-½
C	-1	-1	-½	+1

Needleman-Wunschov algoritmus

- Tento algoritmus vykonáva **globálne zarovnanie**, čo znamená, že hľadá **najlepšie možné zarovnanie dvoch sekvencií po celej ich dĺžke.**
- Líši sa od **lokálneho zarovnávania**, ktoré sa zameriava na hľadanie oblastí s najvyššou podobnosťou (vysvetlíme si neskôr).

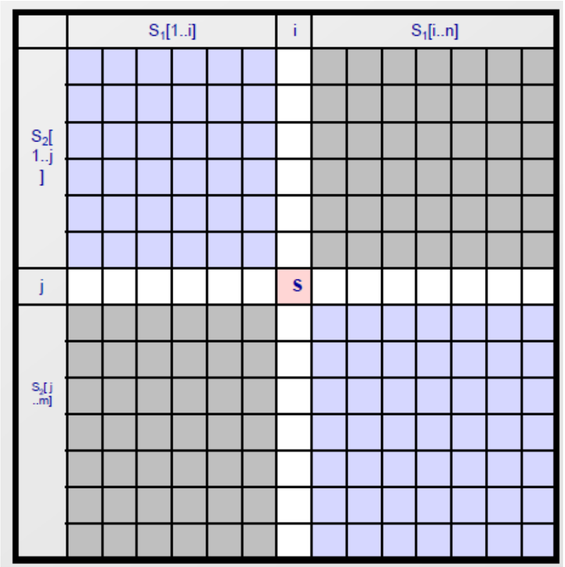
4. The Needleman-Wunsch Algorithm

Definícia problému: Princíp optimálnej podštruktúry

Kľúčový poznatok (The Key Insight)

Ak máme optimálne zarovnanie sekvencií $S_{1..n}$ a $T_{1..m}$, v ktorom sa báza S_i zhoduje s T_j, potom platí:

- Toto globálne optimum sa povinne skladá z **optimálneho zarovnania prefixov** $(S_1 \dots S_{i-1}, T_1 \dots T_{j-1})$ a **optimálneho zarovnania sufixov** $(S_{i+1} \dots S_n, T_{j+1} \dots T_m)$.



Argument „vystrihnutia a vloženia“ (Cut-and-paste)

Ak by niektoré z týchto čiastkových zarovnaní nebolo optimálne, mohli by sme ho „vystrihnúť“ a nahradiť lepším (s vyšším skóre). Tým by sme získali vyššie celkové skóre, čo je v priamom rozpore s predpokladom, že pôvodné zarovnanie bolo optimálne.
Dôsledok: Každá pod-cesta v rámci optimálnej cesty musí byť sama o sebe optimálna.

Aditivita a nezávislosť

- **Sčítateľnosť skóre:** Celkové skóre zarovnania je súčtom skóre jednotlivých pod-zarovnaní.
- **Predpoklad nezávislosti:** Tento model predpokladá, že podproblémy výpočtu optimálnych skóre sú navzájom nezávislé.
- **Biologická validácia:** Pre zmysluplné výsledky musíme biologicky zdôvodniť, že mutácie v jednej časti sekvencie neovplyvňujú pravdepodobnosť mutácií v inej časti.

4. The Needleman-Wunsch Algorithm

Matematický model a priestor podproblémov

Základom algoritmu je vytvorenie matice F s rozmermi $(m+1) \times (n+1)$, kde každá bunka $F_{i,j}$ predstavuje optimálne skóre zarovnania prefixov sekvencií S a T. Tento prístup využíva metódu **tabulácie (bottom-up)**, kde sa zložité globálne zarovnanie rozkladá na menšie, lokálne závislé kroky.

Inicializácia a výpočet:

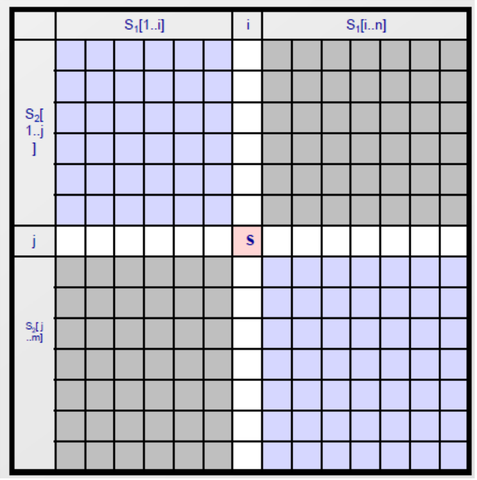
- **Okrajové podmienky:** Prvý riadok a stĺpec reprezentujú zarovnanie proti prázdnemu reťazcu, preto sa inicializujú kumulatívnou penalizáciou za medzeru: $F(i,0) = -i \cdot d$ a $F(0,j) = -j \cdot d$.
- **Štyri biologické scenáre:** V každom kroku algoritmus vyhodnocuje zhodu (match), mutáciu (substitution), inzerciu alebo deléciu (gaps).
- **Rekurzívne pravidlo:** Hodnota každej bunky je určená maximom z troch možných smerov:

$$
\begin{aligned}
Initialization \quad &: \quad \begin{array}{l} F(0,0) = 0 \\ F(i,0) = F(i-1,0) - d \\ F(0,j) = F(0,j-1) - d \end{array} \\
Iteration \quad &: \quad F(i,j) = \max \begin{cases} F(i-1,j) - d & \text{insert gap in } S \\ F(i,j-1) - d & \text{insert gap in } T \\ F(i-1,j-1) + s(x_i, y_j) & \text{match or mutation} \end{cases} \\
Termination \quad &: \quad Bottom\ right
\end{aligned}
$$

4. The Needleman-Wunsch Algorithm

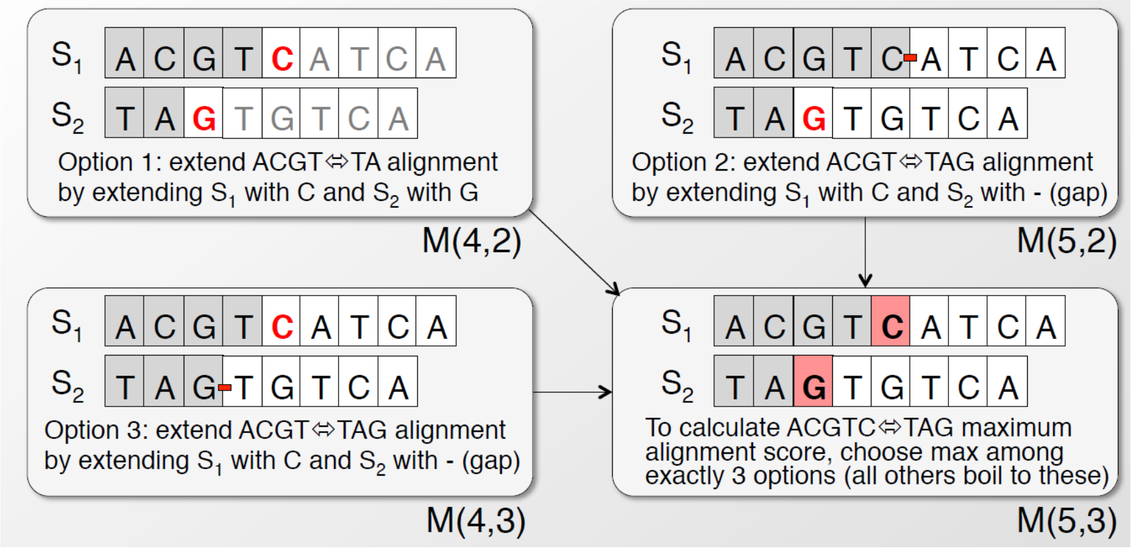
Proces prechádzania matice a spätné sledovanie (Traceback)

Aby bol algoritmus efektívny, matica sa prechádza v takom poradí, aby pri výpočte ***F(i,j)*** boli hodnoty vľavo, hore a diagonálne vľavo hore už známe. Po dosiahnutí pravého dolného rohu matice (***F_{m,n}***) získavame finálne optimálne skóre celého zarovnania.



Získanie finálneho zarovnania:

- **Ukladanie rozhodnutí:** Súbežne s výpočtom skóre sa ukladajú smerovníky (pointers), ktoré zaznamenávajú, ktorá z troch ciest viedla k maximálnej hodnote v danej bunke.
- **Traceback:** Proces začína v bunke ***F_{m,n}*** a spätne sleduje cestu k počiatku (0,0). Každý krok spätnej cesty určuje konkrétny stĺpec výsledného zarovnania:
- **Diagonálny pohyb:**
  Priradenie nukleotidu k nukleotidu.
- **Vertikálny/Horizontálny pohyb:**
  Vloženie medzery (gap) do jednej zo sekvencií.



4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie A C G T a A G T C

Vstupné údaje

- **Sekvencia S (vertikálna):** `A C G T`
- **Sekvencia T (horizontálna):** `A G T C`
- **Skórovacia schéma:**
  - Zhoda (Match): **+2**
  - Zámena (Mismatch): **-1**
  - Penalizácia za medzeru (d): **-2**

1. Krok: Inicializácia matice F

Matica F má indexy i (riadky, $0 \ldots 4$) a j (stĺpce, $0 \ldots 4$). Prvý riadok a stĺpec reprezentujú zarovnanie proti medzerám.

- $F_{0,0} = 0$
- $F_{1,0} = F_{0,0} - 2 = \mathbf{-2}$
- $F_{2,0} = F_{1,0} - 2 = \mathbf{-4}$
- $F_{3,0} = -6, F_{4,0} = -8$
- $F_{0,1} = -2, F_{0,2} = -4, F_{0,3} = -6, F_{0,4} = -8$

- $F_{0,0} = 0$: Počiatočný bod (prázdny reťazec vs. prázdny reťazec).
- $F_{0,1}$ **až** $F_{0,4}$: Reprezentujú narastajúci počet medzier v sekvencii S. Výpočet: $F_{0,j} = F_{0,j-1} + d$.
- $F_{1,0}$ **až** $F_{4,0}$: Reprezentujú narastajúci počet medzier v sekvencii T. Výpočet: $F_{i,0} = F_{i-1,0} + d$.

Tieto hodnoty tvoria „hranicu“, z ktorej budeme v ďalšom kroku odvodzovať hodnoty pre vnútro matice (označené ako **?**).

1. Krok: Inicializácia matice F

Predtým, než začneme porovnávať samotné bázy, musíme maticu pripraviť. Prvý riadok ($i = 0$) a prvý stĺpec ($j = 0$) reprezentujú situáciu, kedy jednu sekvenciu zarovnávame proti čistým medzerám (gapom) v druhej sekvencii.

Každý krok smerom doprava alebo dole stojí **-2** body (podľa našej schémy $d = -2$).

Čiastočne vyplnená tabuľka (Stav po inicializácii):

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	?	?	?	?
C ($i = 2$)	$F_{2,0} = -4$	?	?	?	?
G ($i = 3$)	$F_{3,0} = -6$	?	?	?	?
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie A C G T a A G T C

Pripomenutie skórovacej schémy:

- **Zhoda (Match):** $+2$
- **Zámena (Mismatch):** -1
- **Medzera (Gap):** -2

Výpočet 1. riadku: Báza S[1] = A

V tomto riadku porovnávame prvú bázu sekvencie S (**A**) so všetkými bázami sekvencie T (**A, G, T, C**). Využívame hodnoty z inicializačného (nultého) riadku a stĺpca.

- $F_{1,1}$ **(A vs. A):**
  - Diagonála ($F_{0,0}$ + zhoda): $0 + 2 = \mathbf{2}$
  - Zhora ($F_{0,1} - 2$): $-2 - 2 = -4$
  - Zľava ($F_{1,0} - 2$): $-2 - 2 = -4$
  - **Výsledok** $F_{1,1} = \max(2, -4, -4) = 2$ (Smer: Diagonálne - zhoda)
- $F_{1,2}$ **(A vs. G):**
  - Diagonála ($F_{0,1}$ + zámena): $-2 + (-1) = -3$
  - Zhora ($F_{0,2} - 2$): $-4 - 2 = -6$
  - Zľava ($F_{1,1} - 2$): $2 - 2 = \mathbf{0}$
  - **Výsledok** $F_{1,2} = \max(-3, -6, 0) = 0$ (Smer: Zľava - medzera v S)
- $F_{1,3}$ **(A vs. T):**
  - Diagonála ($F_{0,2}$ + zámena): $-4 + (-1) = -5$
  - Zhora ($F_{0,3} - 2$): $-6 - 2 = -8$
  - Zľava ($F_{1,2} - 2$): $0 - 2 = \mathbf{-2}$
  - **Výsledok** $F_{1,3} = \max(-5, -8, -2) = -2$ (Smer: Zľava - medzera v S)
- $F_{1,4}$ **(A vs. C):**
  - Diagonála ($F_{0,3}$ + zámena): $-6 + (-1) = -7$
  - Zhora ($F_{0,4} - 2$): $-8 - 2 = -10$
  - Zľava ($F_{1,3} - 2$): $-2 - 2 = \mathbf{-4}$
  - **Výsledok** $F_{1,4} = \max(-7, -10, -4) = -4$ (Smer: Zľava - medzera v S)

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	$F_{1,1} = 2$	$F_{1,2} = 0$	$F_{1,3} = -2$	$F_{1,4} = -4$
C ($i = 2$)	$F_{2,0} = -4$	?	?	?	?
G ($i = 3$)	$F_{3,0} = -6$	?	?	?	?
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie A C G T a A G T C

Výpočet 2. riadku: Báza S[2] = C

V tomto riadku porovnávame **C** postupne s bázami sekvencie T (A, G, T, C).

- $F_{2,1}$ **(C vs. A):**
  - Diagonála ($F_{1,0}$ + zámena): $-2 + (-1) = -3$
  - Zhora ($F_{1,1} - 2$): $2 - 2 = 0$
  - Zľava ($F_{2,0} - 2$): $-4 - 2 = -6$
  - **Výsledok** $F_{2,1} = \max(-3, 0, -6) = 0$ (Smer: Zhora)
- $F_{2,2}$ **(C vs. G):**
  - Diagonála ($F_{1,1}$ + zámena): $2 + (-1) = 1$
  - Zhora ($F_{1,2} - 2$): $0 - 2 = -2$
  - Zľava ($F_{2,1} - 2$): $0 - 2 = -2$
  - **Výsledok** $F_{2,2} = \max(1, -2, -2) = 1$ (Smer: Diagonálne)
- $F_{2,3}$ **(C vs. T):**
  - Diagonála ($F_{1,2}$ + zámena): $0 + (-1) = -1$
  - Zhora ($F_{1,3} - 2$): $-2 - 2 = -4$
  - Zľava ($F_{2,2} - 2$): $1 - 2 = -1$
  - **Výsledok** $F_{2,3} = \max(-1, -4, -1) = -1$ (Smer: Diagonálne alebo Zľava)
- $F_{2,4}$ **(C vs. C):**
  - Diagonála ($F_{1,3}$ + zhoda): $-2 + 2 = 0$
  - Zhora ($F_{1,4} - 2$): $-4 - 2 = -6$
  - Zľava ($F_{2,3} - 2$): $-1 - 2 = -3$
  - **Výsledok** $F_{2,4} = \max(0, -6, -3) = 0$ (Smer: Diagonálne)

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	$F_{1,1} = 2$	$F_{1,2} = 0$	$F_{1,3} = -2$	$F_{1,4} = -4$
C ($i = 2$)	$F_{2,0} = -4$	?	?	?	?
G ($i = 3$)	$F_{3,0} = -6$	?	?	?	?
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	$F_{1,1} = 2$	$F_{1,2} = 0$	$F_{1,3} = -2$	$F_{1,4} = -4$
C ($i = 2$)	$F_{2,0} = -4$	$F_{2,1} = 0$	$F_{2,2} = 1$	$F_{2,3} = -1$	$F_{2,4} = 0$
G ($i = 3$)	$F_{3,0} = -6$	?	?	?	?
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

4. The Needleman-Wunsch Algorithm

Výpočet 3. riadku: Báza S[3] = G

Teraz porovnávame **G** postupne s bázami T (A, G, T, C) s využitím hodnôt z práve vypočítaného 2. riadku.

- $F_{3,1}$ **(G vs. A):**
  - Diagonála ($F_{2,0} + \text{zámena}$): $-4 + (-1) = -5$
  - Zhora ($F_{2,1} - 2$): $0 - 2 = -2$
  - Zľava ($F_{3,0} - 2$): $-6 - 2 = -8$
  - **Výsledok** $F_{3,1} = \max(-5, -2, -8) = -2$ (Smer: Zhora)
- $F_{3,2}$ **(G vs. G):**
  - Diagonála ($F_{2,1} + \text{zhoda}$): $0 + 2 = 2$
  - Zhora ($F_{2,2} - 2$): $1 - 2 = -1$
  - Zľava ($F_{3,1} - 2$): $-2 - 2 = -4$
  - **Výsledok** $F_{3,2} = \max(2, -1, -4) = 2$ (Smer: Diagonálne)
- $F_{3,3}$ **(G vs. T):**
  - Diagonála ($F_{2,2} + \text{zámena}$): $1 + (-1) = 0$
  - Zhora ($F_{2,3} - 2$): $-1 - 2 = -3$
  - Zľava ($F_{3,2} - 2$): $2 - 2 = 0$
  - **Výsledok** $F_{3,3} = \max(0, -3, 0) = 0$ (Smer: Diagonálne alebo Zľava)
- $F_{3,4}$ **(G vs. C):**
  - Diagonála ($F_{2,3} + \text{zámena}$): $-1 + (-1) = -2$
  - Zhora ($F_{2,4} - 2$): $0 - 2 = -2$
  - Zľava ($F_{3,3} - 2$): $0 - 2 = -2$
  - **Výsledok** $F_{3,4} = \max(-2, -2, -2) = -2$ (Všetky smery sú rovnocenné)

Príklad: Zarovnajte sekvencie A C G T a A G T C

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	$F_{1,1} = 2$	$F_{1,2} = 0$	$F_{1,3} = -2$	$F_{1,4} = -4$
C ($i = 2$)	$F_{2,0} = -4$	$F_{2,1} = 0$	$F_{2,2} = 1$	$F_{2,3} = -1$	$F_{2,4} = 0$
G ($i = 3$)	$F_{3,0} = -6$	?	?	?	?
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	$F_{0,0} = 0$	$F_{0,1} = -2$	$F_{0,2} = -4$	$F_{0,3} = -6$	$F_{0,4} = -8$
A ($i = 1$)	$F_{1,0} = -2$	$F_{1,1} = 2$	$F_{1,2} = 0$	$F_{1,3} = -2$	$F_{1,4} = -4$
C ($i = 2$)	$F_{2,0} = -4$	$F_{2,1} = 0$	$F_{2,2} = 1$	$F_{2,3} = -1$	$F_{2,4} = 0$
G ($i = 3$)	$F_{3,0} = -6$	$F_{3,1} = -2$	$F_{3,2} = 2$	$F_{3,3} = 0$	$F_{3,4} = -2$
T ($i = 4$)	$F_{4,0} = -8$	?	?	?	?

4. The Needleman-Wunsch Algorithm

Pripomenutie skórovacej schémy: **Zhoda = +2, Zámena = -1, Medzera = -2**.
Využívame hodnoty z predchádzajúceho (tretieho) riadku:
$F_{3,0} = -6, F_{3,1} = -2, F_{3,2} = 2, F_{3,3} = 0, F_{3,4} = -2.$

Výpočet 4. riadku: Báza S[4] = T

V tomto riadku porovnávame **T** postupne s bázami sekvencie T (A, G, T, C).

- $F_{4,1}$ **(T vs. A):**
  - Diagonála ($F_{3,0}$ + zámena): $-6 + (-1) = -7$
  - Zhora ($F_{3,1} - 2$): $-2 - 2 = -4$
  - Zľava ($F_{4,0} - 2$): $-8 - 2 = -10$
  - **Výsledok** $F_{4,1} = \max(-7, -4, -10) = -4$ (Smer: Zhora)
- $F_{4,2}$ **(T vs. G):**
  - Diagonála ($F_{3,1}$ + zámena): $-2 + (-1) = -3$
  - Zhora ($F_{3,2} - 2$): $2 - 2 = 0$
  - Zľava ($F_{4,1} - 2$): $-4 - 2 = -6$
  - **Výsledok** $F_{4,2} = \max(-3, 0, -6) = 0$ (Smer: Zhora)

Príklad: Zarovnajte sekvencie A C G T a A G T C

- $F_{4,3}$ **(T vs. T):**
  - Diagonála ($F_{3,2}$ + zhoda): $2 + 2 = \mathbf{4}$
  - Zhora ($F_{3,3} - 2$): $0 - 2 = -2$
  - Zľava ($F_{4,2} - 2$): $0 - 2 = -2$
  - **Výsledok** $F_{4,3} = \max(4, -2, -2) = 4$ (Smer: Diagonálne - zhoda)
- $F_{4,4}$ **(T vs. C):**
  - Diagonála ($F_{3,3}$ + zámena): $0 + (-1) = -1$
  - Zhora ($F_{3,4} - 2$): $-2 - 2 = -4$
  - Zľava ($F_{4,3} - 2$): $4 - 2 = \mathbf{2}$
  - **Výsledok** $F_{4,4} = \max(-1, -4, 2) = 2$ (Smer: Zľava - medzera v S)

	-	**A** ($j = 1$)	**G** ($j = 2$)	**T** ($j = 3$)	**C** ($j = 4$)
-	**0**	-2	-4	-6	-8
A ($i = 1$)	-2	**2**	0	-2	-4
C ($i = 2$)	-4	0	1	-1	0
G ($i = 3$)	-6	-2	**2**	0	-2
T ($i = 4$)	-8	-4	0	**4**	**2**

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie A C G T a A G T C

Toto je kompletná tabuľka so všetkými vypočítanými hodnotami. Číslo v pravom dolnom rohu (**2**) je optimálne skóre globálneho zarovnania týchto dvoch sekvencií.

	-	A ($j = 1$)	G ($j = 2$)	T ($j = 3$)	C ($j = 4$)
-	**0**	-2	-4	-6	-8
A ($i = 1$)	-2	**2**	0	-2	-4
C ($i = 2$)	-4	0	1	-1	0
G ($i = 3$)	-6	-2	**2**	0	-2
T ($i = 4$)	-8	-4	0	**4**	**2**

```
S: A C G T -
T: A - G T C
```

Interpretácia výsledku (Traceback)

Ak chceme získať samotné zarovnanie, ideme od $F_{4,4} = 2$ spätne:

1. Z **2** ($F_{4,4}$) sme prišli zľava z hodnoty **4** ($F_{4,3}$). To znamená, že posledná báza sekvencie T (**C**) je zarovnaná s medzerou v sekvencii S.
2. Z **4** ($F_{4,3}$) sme prišli diagonálne z hodnoty **2** ($F_{3,2}$). To je zhoda **T** s **T**.
3. Z **2** ($F_{3,2}$) sme prišli diagonálne z hodnoty **0** ($F_{2,1}$). To je zhoda **G** s **G**.
   (Všimnite si: v našom detailnom výpočte 2. riadku sme určili $F_{2,1} = 0$, čo umožnilo túto cestu).
4. Z **0** ($F_{2,1}$) sme prišli zhora z hodnoty **2** ($F_{1,1}$). To znamená, že báza **C** zo sekvencie S je zarovnaná s medzerou v sekvencii T.
5. Z **2** ($F_{1,1}$) sme prišli diagonálne z **0** ($F_{0,0}$). To je zhoda **A** s **A**.

Skóre: 2 (A-A) - 2 (gap) + 2 (G-G) + 2 (T-T) - 2 (gap) = **2**.

Tento matematický postup zaručuje, že neexistuje žiadne iné zarovnanie s vyšším skóre ako 2.

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Pri afinnej penalizácii (Affine Gap Penalty) je výpočet zložitejší, pretože si musíme pamätať, či medzeru práve **otvárame** (vysoká penalizácia), alebo v nej **pokračujeme** (nízka penalizácia).

V praxi sa to počíta pomocou troch matíc (M pre zhody, I pre inzercie, D pre delécie), ale pre tento podrobný príklad budeme simulovať rozhodovanie v rámci jednej tabuľky F so zjednodušeným pravidlom:

- **Zhoda/Zámena:** $+3$ / -1
- **Otvorenie medzery (gap_{open}):** -5
- **Predĺženie medzery (gap_{ext}):** -1

Sekvencie: $S = \mathrm{GATC}$ (vertikálne), $T = \mathrm{GACT}$ (horizontálne).

Krok 1: Inicializácia (Riadok 0)

Prvá medzera stojí -5, každá ďalšia už len -1.

- $F_{0,0} = 0$
- $F_{0,1} = -5$ (otvorenie)
- $F_{0,2} = -5 - 1 = -6$ (predĺženie)
- $F_{0,3} = -7, F_{0,4} = -8$
- To isté platí pre prvý stĺpec ($F_{1,0} = -5, F_{2,0} = -6, \ldots$)

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	?	?	?	?
A	-6	?	?	?	?
T	-7	?	?	?	?
C	-8	?	?	?	?

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Krok 2: Výpočet 1. riadku (S[1] = G)

- $F_{1,1}$ **(G-G):** Diagonála ($0 + 3 = 3$), Zhora ($-5 - 5 = -10$), Zľava ($-5 - 5 = -10$).
  Výsledok: 3 (Zhoda).
- $F_{1,2}$ **(G-A):** Diagonála ($-5 - 1 = -6$), Zhora ($-6 - 5 = -11$), Zľava ($3 - 5 = -2$).
  Výsledok: -2 (Medzera v S).
- $F_{1,3}$ **(G-C):** Diagonála ($-6 - 1 = -7$), Zhora ($-7 - 5 = -12$), Zľava ($-2 - 1 = -3$).
  Výsledok: -3 (Predĺženie medzery v S).
- $F_{1,4}$ **(G-T):** Diagonála ($-7 - 1 = -8$), Zhora ($-8 - 5 = -13$), Zľava ($-3 - 1 = -4$).
  Výsledok: -4 (Predĺženie medzery).

Tabuľka po 1. riadku:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	**3**	**-2**	**-3**	**-4**

Krok 3: Výpočet 2. riadku (S[2] = A)

- $F_{2,1}$ **(A-G):** Diagonála ($-5 - 1 = -6$), Zhora ($3 - 5 = -2$), Zľava ($-6 - 5 = -11$).
  Výsledok: -2 (Medzera v T).
- $F_{2,2}$ **(A-A):** Diagonála ($3 + 3 = 6$), Zhora ($-2 - 5 = -7$), Zľava ($-2 - 5 = -7$).
  Výsledok: 6 (Zhoda).
- $F_{2,3}$ **(A-C):** Diagonála ($-3 - 1 = -4$), Zhora ($-3 - 5 = -8$), Zľava ($6 - 5 = 1$).
  Výsledok: 1 (Medzera v S).
- $F_{2,4}$ **(A-T):** Diagonála ($-4 - 1 = -5$), Zhora ($-4 - 5 = -9$), Zľava ($1 - 1 = 0$).
  Výsledok: 0 (Predĺženie medzery).

Tabuľka po 2. riadku:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	3	-2	-3	-4
A	-6	**-2**	**6**	**1**	**0**

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Krok 4: Výpočet 3. riadku (S[3] = T)

- $F_{3,1}$ **(T-G):** Diagonála ($-6-1=-7$), Zhora ($-2-5=-7$), Zľava ($-7-5=-12$). **Výsledok: -7**.
- $F_{3,2}$ **(T-A):** Diagonála ($-2-1=-3$), Zhora ($6-5=1$), Zľava ($-7-5=-12$). **Výsledok: 1**.
- $F_{3,3}$ **(T-C):** Diagonála ($6-1=5$), Zhora ($1-5=-4$), Zľava ($1-5=-4$). **Výsledok: 5** (Zámena, ale výhodná z predchádzajúcej zhody).
- $F_{3,4}$ **(T-T):** Diagonála ($1+3=4$), Zhora ($0-5=-5$), Zľava ($5-5=0$). **Výsledok: 4** (Zhoda).

Tabuľka po 3. riadku:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	3	-2	-3	-4
A	-6	-2	6	1	0
T	-7	**-7**	**1**	**5**	**4**

Krok 5: Výpočet 4. riadku (S[4] = C)

- $F_{4,1}$ **(C-G):** Diagonála ($-7-1=-8$), Zhora ($-7-5=-12$), Zľava ($-8-5=-13$). **Výsledok: -8**.
- $F_{4,2}$ **(C-A):** Diagonála ($-7-1=-8$), Zhora ($1-5=-4$), Zľava ($-8-5=-13$). **Výsledok: -4**.
- $F_{4,3}$ **(C-C):** Diagonála ($1+3=4$), Zhora ($5-5=0$), Zľava ($-4-5=-9$). **Výsledok: 4** (Zhoda).
- $F_{4,4}$ **(C-T):** Diagonála ($5-1=4$), Zhora ($4-5=-1$), Zľava ($4-5=-1$). **Výsledok: 4** (Zámena).

Výsledná finálna tabuľka:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	**3**	-2	-3	-4
A	-6	-2	**6**	1	0
T	-7	-7	1	**5**	4
C	-8	-8	-4	**4**	**4**

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Krok 4: Výpočet 3. riadku (S[3] = T)

- $F_{3,1}$ **(T-G):** Diagonála ($-6-1=-7$), Zhora ($-2-5=-7$), Zľava ($-7-5=-12$). **Výsledok: -7**.
- $F_{3,2}$ **(T-A):** Diagonála ($-2-1=-3$), Zhora ($6-5=1$), Zľava ($-7-5=-12$). **Výsledok: 1**.
- $F_{3,3}$ **(T-C):** Diagonála ($6-1=5$), Zhora ($1-5=-4$), Zľava ($1-5=-4$). **Výsledok: 5** (Zámena, ale výhodná z predchádzajúcej zhody).
- $F_{3,4}$ **(T-T):** Diagonála ($1+3=4$), Zhora ($0-5=-5$), Zľava ($5-5=0$). **Výsledok: 4** (Zhoda).

Tabuľka po 3. riadku:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	3	-2	-3	-4
A	-6	-2	6	1	0
T	-7	**-7**	**1**	**5**	**4**

Krok 5: Výpočet 4. riadku (S[4] = C)

- $F_{4,1}$ **(C-G):** Diagonála ($-7-1=-8$), Zhora ($-7-5=-12$), Zľava ($-8-5=-13$). **Výsledok: -8**.
- $F_{4,2}$ **(C-A):** Diagonála ($-7-1=-8$), Zhora ($1-5=-4$), Zľava ($-8-5=-13$). **Výsledok: -4**.
- $F_{4,3}$ **(C-C):** Diagonála ($1+3=4$), Zhora ($5-5=0$), Zľava ($-4-5=-9$). **Výsledok: 4** (Zhoda).
- $F_{4,4}$ **(C-T):** Diagonála ($5-1=4$), Zhora ($4-5=-1$), Zľava ($4-5=-1$). **Výsledok: 4** (Zámena).

Výsledná finálna tabuľka:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	**3**	-2	-3	-4
A	-6	-2	**6**	1	0
T	-7	-7	1	**5**	4
C	-8	-8	-4	**4**	**4**

Finálne skóre: 4. Optimálne zarovnanie by bolo:

```
S: G A T C
T: G A C T
```

Zhoda G, Zhoda A, Zámena T/C, Zámena C/T.
Skóre: 3+3-1-1 = 4.

Všimnite si, že algoritmus s afinnou penalizáciou by pri dlhších sekvenciách radšej zvolil jednu zámenu (-1), než by otváral novú medzeru (-5).

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Krok 4: Výpočet 3. riadku (S[3] = T)

- $F_{3,1}$ **(T-G):** Diagonála ($-6-1=-7$), Zhora ($-2-5=-7$), Zľava ($-7-5=-12$). **Výsledok: -7**.
- $F_{3,2}$ **(T-A):** Diagonála ($-2-1=-3$), Zhora ($6-5=1$), Zľava ($-7-5=-12$). **Výsledok: 1**.
- $F_{3,3}$ **(T-C):** Diagonála ($6-1=5$), Zhora ($1-5=-4$), Zľava ($1-5=-4$). **Výsledok: 5** (Zámena, ale výhodná z predchádzajúcej zhody).
- $F_{3,4}$ **(T-T):** Diagonála ($1+3=4$), Zhora ($0-5=-5$), Zľava ($5-5=0$). **Výsledok: 4** (Zhoda).

Tabuľka po 3. riadku:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	3	-2	-3	-4
A	-6	-2	6	1	0
T	-7	**-7**	**1**	**5**	**4**

Krok 5: Výpočet 4. riadku (S[4] = C)

- $F_{4,1}$ **(C-G):** Diagonála ($-7-1=-8$), Zhora ($-7-5=-12$), Zľava ($-8-5=-13$). **Výsledok: -8**.
- $F_{4,2}$ **(C-A):** Diagonála ($-7-1=-8$), Zhora ($1-5=-4$), Zľava ($-8-5=-13$). **Výsledok: -4**.
- $F_{4,3}$ **(C-C):** Diagonála ($1+3=4$), Zhora ($5-5=0$), Zľava ($-4-5=-9$). **Výsledok: 4** (Zhoda).
- $F_{4,4}$ **(C-T):** Diagonála ($5-1=4$), Zhora ($4-5=-1$), Zľava ($4-5=-1$). **Výsledok: 4** (Zámena).

Výsledná finálna tabuľka:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	**3**	-2	-3	-4
A	-6	-2	**6**	1	0
T	-7	-7	1	**5**	4
C	-8	-8	-4	**4**	**4**

Finálne skóre: 4. Optimálne zarovnanie by bolo:

```
S: G A T C
T: G A C T
```

Zhoda G, Zhoda A, Zámena T/C, Zámena C/T.
Skóre: 3+3-1-1 = 4.

Všimnite si, že algoritmus s afinnou penalizáciou by pri dlhších sekvenciách radšej zvolil jednu zámenu (-1), než by otváral novú medzeru (-5).

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Teraz príklad zmeníme a ukážeme si ako sa zmení dynamika výpočtu, ak znížime penalizáciu za otvorenie medzery (gap_{open}) z -5 na -3.

Ostatné parametre zostávajú rovnaké: zhoda +3, zámena -1, predĺženie medzery -1.
S nižšou penalizáciou za otvorenie medzery sa algoritmus stáva „odvážnejším“ pri vkladaní medzier, pretože už nie sú také drahé v porovnaní so zámenou nukleotidov.

1. Inicializácia (Riadok 0)

- $F_{0,0} = 0$
- $F_{0,1} = -3$ (otvorenie)
- $F_{0,2} = -4, F_{0,3} = -5, F_{0,4} = -6$ (každé predĺženie stojí -1)

	-	G	A	C	T
-	0	-3	-4	-5	-6
G	-3	?	?	?	?

Stav po 1. riadku (S[1] = G)

V bunke $F_{1,2}$ vidíme zmenu: $3 - 3 = 0$. Pri predchádzajúcom zadaní by to bolo $3 - 5 = -2$.

	-	G	A	C	T
-	0	-3	-4	-5	-6
G	-3	**3**	**0**	**-1**	**-2**

Stav po 2. riadku (S[2] = A)

Vďaka lacnejším medzerám sa skóre v tomto riadku drží vyššie.

	-	G	A	C	T
-	0	-3	-4	-5	-6
G	-3	3	0	-1	-2
A	-4	0	**6**	**3**	**2**

Stav po 3. riadku (S[3] = T)

Všimnite si bunku $F_{3,4}$: Zhoda **T-T** sa pripočíta k $F_{2,3}$ $(3 + 3 = 6)$.

	-	G	A	C	T
-	0	-3	-4	-5	-6
G	-3	3	0	-1	-2
A	-4	0	6	3	2
T	-5	-1	3	5	**6**

4. The Needleman-Wunsch Algorithm

Príklad: Zarovnajte sekvencie S = GATC (vertikálne), T = GACT (horizontálne). Použite afinnú penalizáciu (Affine Gap Penalty)

Teraz príklad zmeníme a ukážeme si ako sa zmení dynamika výpočtu, ak znížime penalizáciu za otvorenie medzery (gap_{open}) z -5 na -3.

Ostatné parametre zostávajú rovnaké: zhoda +3, zámena -1, predĺženie medzery -1.
S nižšou penalizáciou za otvorenie medzery sa algoritmus stáva „odvážnejším“ pri vkladaní medzier, pretože už nie sú také drahé v porovnaní so zámenou nukleotidov.

(gap_{open})= - 3

3. Finálna tabuľka (S[4] = C)

	-	G	A	C	T
-	0	-3	-4	-5	-6
G	-3	3	0	-1	-2
A	-4	0	6	3	2
T	-5	-1	3	5	6
C	-6	-2	2	**6**	**5**

(gap_{open})= - 5

Výsledná finálna tabuľka:

	-	G	A	C	T
-	0	-5	-6	-7	-8
G	-5	**3**	-2	-3	-4
A	-6	-2	**6**	1	0
T	-7	-7	1	**5**	4
C	-8	-8	-4	**4**	**4**

Kľúčový rozdiel a výsledok

- **Pôvodné skóre (gap -5): 4**
- **Nové skóre (gap -3): 6**

Zníženie ceny za otvorenie medzery zmenilo optimálnu cestu. Kým predtým algoritmus akceptoval dve zámeny (T-C a C-T), teraz zistil, že je výhodnejšie „roztrhnúť“ sekvenciu medzerami, aby dosiahol viac zhôd.